

Caching

- * Storing frequently used data in a fast-access storage layer so future requests can be served quickly.
- * Cache is usually stored in:
 - 1) In memory systems (Redis, memcached)
 - 2) Application memory
 - 3) Content delivery networks (for static content)

Working

- * User requests data → Backend checks ~~data~~ cache
- * If found (cache hit) → returns immediately
- * If not found (cache miss) → fetch from DB → store in cache → return.

What happens without caching?

- System under heavy load:
 - DB CPU spikes
 - Queries slow down
 - Timeouts occur
 - System crashes.

caching is often first scaling solution.

Trade offs

Pros

- Faster responses
- Reduced DB load
- Better scalability

- Cache invalidation is one of the hardest problems in system design.

Cons

- Stale data risk
- Cache invalidation complexity
- Extra infrastructure.

Caching strategies

1) Cache - Aside (Lazy load) → most common working

- Application checks cache → If cache hit → return data
- If cache miss → fetch from DB → store in cache → return response.

On update:

- Update db
- Invalidate or update cache manually.

Note:- First request after expiry is slow.

2) Write - Through Cache → when writing data:

→ write to cache & DB (Both happen together)

- * It ensures cache always has fresh data (no stale reads)

Note:- slower writes & cache may store unused data.

3) Write ~~Back~~ Back → write to cache then cache updates DB asynchronously.

→ very fast writes & reduces DB load.

Note:- If cache crashes before DB update → Data loss

→ Best for non-critical workloads & high throughput systems

4) Time Based Expiration (TTL) →

Each cached item has time to live, after expiration cache invalidates automatically.

→ Data may be stale until TTL expires.

→ Good for data that changes infrequently.

Common caching mistakes

- Forgetting to invalidate data.
- Using very long TTL for dynamic data.
- Caching highly dynamic data unnecessarily

1) Banking → use write-through

2) Social media feed → Cache aside + event invalidation (Study)

Redis

- * An in-memory data store used as a cache, db. & message broker.
- * It stores data in memory (RAM), not disk.
- Why :-
 - Very fast reads & writes
 - Temporary data storage
 - Real-time counters.
 - Rate limiting
 - Session storage

Persistence → Even it is in-memory, it supports

- * RDB snapshots
 - Append only file
- So data can survive restarts. But not as durable as full relational DB.

Scaling :-

- Supports replication (Primary + replica)
- Clustering (sharding)
- High availability.

Use cases

- 1) Caching → store product list, user profile, API response.
- 2) store user login sessions, instead of application storage.
in Redis
- 3) Rate limiting :- Allow only 100 req. per minute, If counter exceeds → block.
- 4) Real time counters :- Likes, views, notification count.
Handles millions of increments efficiently
- 5) Distributed Locks
used to prevent :-
 - Duplicate job execution
 - Race conditions.

Content Delivery Network

- * A distributed network of servers located in different geographical regions that deliver content to users from the nearest location.
- * Suppose your backend server is in Delhi.
 - Users from: Mumbai, Tokyo, NY all request images, CSS, Javascript.
 - If all traffic goes to Delhi, slow loading, high latency, server overload and bandwidth issues.

Working

- User in London requests :- google.com / image.png
- Instead of going to origin server in Delhi (Suppose):
 - Request comes to nearest CDN edge server (London)
 - If content exists there → server immediately.
 - If not → fetch from origin → cache it → serve
- Next request will be served instantly.

Security

- * CDN provides DDoS protection, Rate limiting, TLS termination, Web Application Firewall (WAF).

CDN Providers

- | | | |
|---------------|---------------------|-------------------|
| 1) Cloudflare | 2) Akamai | 3) AWS CloudFront |
| 4) Fastly | 5) Google Cloud CDN | |

When to Use?

- when you serve static assets
- You have global users
- Traffic is high
- Performance is critical.